

搭配使用 Confidential Space 與未儲存於雲端服務供應商的受保護資源

來源：<https://codelabs.developers.google.com/confidential-space-pki?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將瞭解如何安全共用多方資料，同時透過機密空間維持機密性。本程式碼研究室著重於如何搭配非 Google Cloud 託管的受保護資源使用機密空間。您將瞭解如何提供 Nonce、目標對象和 PKI 權杖類型，向 Google Attestation Service 要求自訂權杖。

目錄

1. [總覽](#)
2. [設定雲端資源](#)
3. [下載根憑證](#)
4. [建立工作負載服務帳戶](#)
5. [建立 USleep 工作負載](#)
6. [建立 UWear 工作負載](#)
7. [執行 USleep 和 UWear 工作負載](#)
8. [\(選用\) 執行未經授權的工作負載](#)
9. [清除](#)

1. 總覽

[Confidential Space](#) 提供安全的資料共用和協作功能，讓機構能與多方分享資料，同時維持資料的機密性。也就是說，機構可以相互協作，同時保有資料控制權，並防止未經授權的存取。

Confidential Space 可讓您匯總及分析機密 (通常是受管制) 資料，從中獲得價值，同時保有完整控制權。有了 Confidential Space，機構就能匯總及分析機密資料 (例如個人識別資訊 (PII)、受保護的健康資訊 (PHI)、智慧財產和加密密鑰) 來獲得價值，讓雙方都受益，同時保有完整控制權。

軟硬體需求

- 兩個不同的 Google Cloud Platform 專案
- [Chrome](#) 或 [Firefox](#) 瀏覽器
- [Google Compute Engine](#)、[機密 VM](#)、[容器](#)、遠端存放區、憑證和憑證鏈結的基本知識。
- 對[服務帳戶](#)、[Open Policy Agent](#)、[Rego](#) 和[公用金鑰基礎架構](#)有基本瞭解

課程內容

- 如何設定執行 Confidential Space 時所需的 Cloud 資源
- 如何在執行 Confidential Space 映像檔的機密 VM 中執行工作負載
- 如何根據工作負載程式碼的屬性 (*what*)、Confidential Space 環境 (*where*) 和執行工作負載的帳戶 (*who*)，授權存取受保護的資源。

本程式碼實驗室著重於如何搭配使用 Confidential Space 與託管於 [Google Cloud](#) 以外位置的受保護資源。您將瞭解如何提供 Nonce、目標對象和 PKI 權杖類型，向 Google 認證服務要求自訂的獨立權杖。

為什麼要要求獨立權杖？

可離線驗證的獨立權杖。離線驗證可提供更高的隱私權，且不必每次都擷取公開金鑰來驗證權杖，因此更實用。而是可以根據先前下載的長期根憑證驗證權杖。

簽署的自給式權杖包含憑證鏈結，也就是根憑證、中繼憑證，以及用於簽署的葉憑證陣列。憑證鏈會透過 [x5c](#) 標頭聲明公開在權杖中。

[根憑證可供下載](#)，並應儲存在應用程式中。根憑證用於驗證分葉憑證簽章。如要驗證信任的憑證鏈結，下載的根憑證和 [x5c](#) 標頭中的根憑證必須相同。

在本程式碼研究室中，您將在虛構產品 USleep (容器化應用程式) 和虛構產品 UWear (連線穿戴式裝置) 之間設定 Confidential Space，以計算睡眠品質。UWear 會在安全且隔離的環境 (又稱「受信任的執行環境」或 TEE) 中，與 USleep 分享受保護的健康資訊 (PHI)，確保資料擁有者保有完整隱私權。

UWear 同時是工作負載稽核人員和資料擁有者。工作負載稽核員會檢查正在執行的工作負載中的程式碼，並記錄映像檔摘要。身為資料擁有者，UWear 會編寫驗證邏輯，檢查權杖及其簽章的有效性。這項服務會使用稽核工作負載的映像檔摘要，編寫驗證政策，只允許特定環境中的特定映像檔摘要存取機密資料。

重要事項：

在本程式碼研究室中，我們使用個別的 GCP 專案和 Confidential Space VM 來展示 UWear 工作流程，但這並非必要。這段程式碼可以直接在邊緣裝置上執行，不需要 Google Cloud 雲端專案或 VM。

在本程式碼研究室中，USleep 會部署容器化應用程式。USleep 無法存取機密資料，但可執行獲准存取機密資料的工作負載。

本程式碼研究室包含下列步驟：

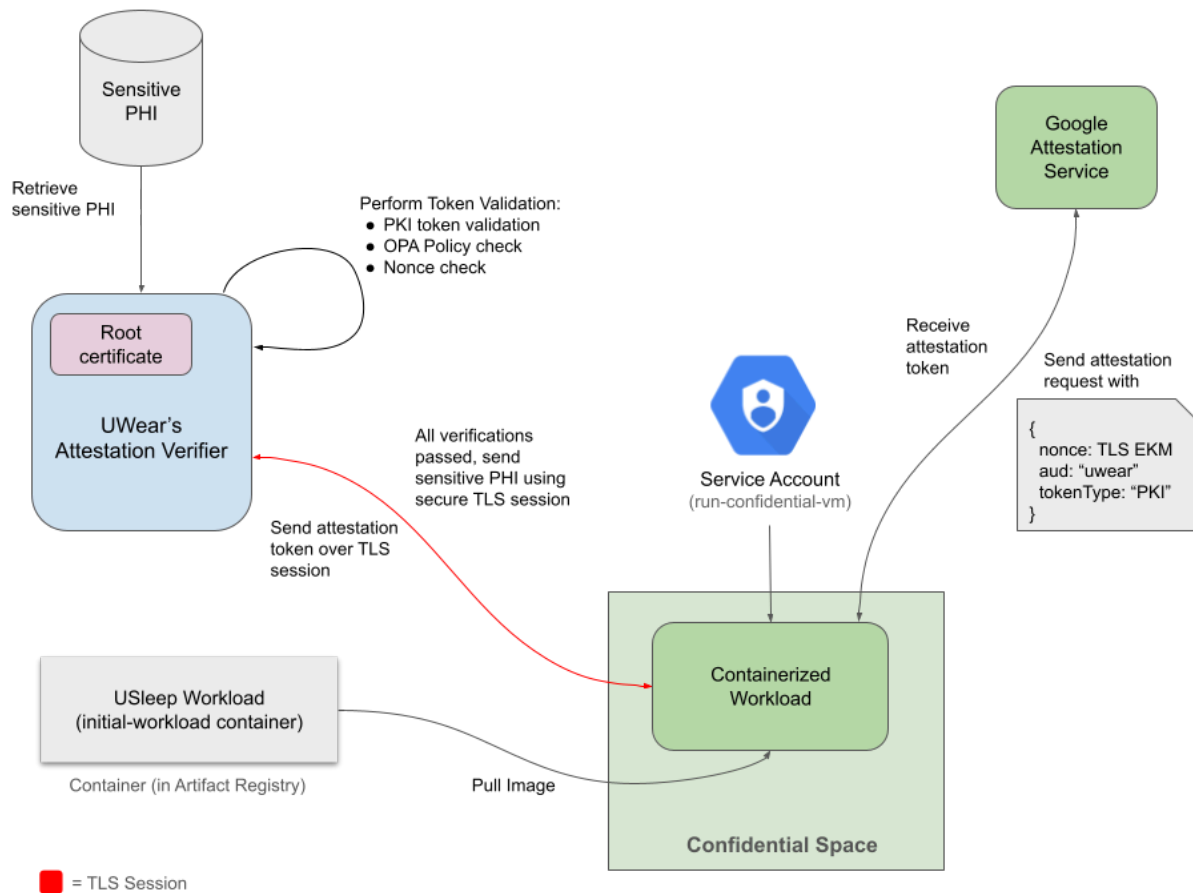
- 步驟 1：為程式碼研究室設定必要的雲端資源。設定專案、帳單和權限。下載程式碼研究室原始碼並設定環境變數。
- 步驟 2：[下載根憑證](#)，並與 UWear 原始碼一起儲存。
- 步驟 3：建立個別的工作負載服務帳戶，供工作負載 VM 用於 USleep 和 UWear。
- 步驟 4：建立提供認證權杖的 USleep 工作負載。
- 步驟 5：建立 UWear 工作負載，驗證認證權杖，並在權杖獲得核准時傳送機密資料。
- 步驟 6：執行 USleep 和 UWear 工作負載。UWear 會提供私密資料，USleep 則會對資料執行睡眠演算法，並輸出結果。
- 步驟 7 (選用)：執行未經授權的 USleep 工作負載，並確認系統未從 UWear 接收機密資料。
- 步驟 8：清理所有資源。

瞭解工作流程

USleep 會在 Confidential Space 中執行工作負載。如要執行工作負載，必須存取 UWear 的 PHI。如要取得存取權，USleep 工作負載會先建立安全的 TLS 工作階段。接著，USleep 也會向 Google 認證服務要求提供認證權杖和[酬載](#)。

USleep 會要求提供認證權杖，並附上 JSON 酬載，其中包含下列三項資訊：

1. **與傳輸層安全標準 (TLS) 工作階段綁定的認證權杖**。如要將認證權杖繫結至 TLS 工作階段，隨機值會是 [TLS 匯出金鑰材料](#)的雜湊值。將權杖繫結至 TLS 工作階段，可確保不會發生中間人攻擊，因為只有參與 TLS 工作階段的雙方可以產生隨機值。
2. **系統會提供「uwear」的目標對象**。UWear 會驗證認證權杖的目標對象是否正確。
3. **「PKI」權杖類型**。「PKI」權杖類型表示 USleep 想要要求獨立權杖。您可以從 [Confidential Space 的知名 PKI 端點](#)下載根憑證，驗證獨立式權杖是否由 Google 簽署。這與預設的 OIDC 權杖類型不同，後者的簽章是使用定期輪替的[公開金鑰](#)驗證。



USleep 工作負載會收到驗證權杖。接著，UWear 會加入與 USleep 的 TLS 連線，並擷取 USleep 的認證權杖。UWear 會根據根憑證檢查 [x5c 聲明](#)，驗證權杖。

如果符合下列條件，UWear 就會核准 USleep 工作負載：

1. 權杖通過 [PKI 驗證邏輯](#)。
2. UWear 會比對 x5c 聲明與根憑證，檢查權杖是否由分葉憑證簽署，最後確認下載的根憑證與 x5c 聲明中的根憑證相同，藉此驗證權杖。
3. 權杖中的工作負載測量聲明符合 [OPA 政策](#)中指定的屬性條件。OPA 是開放原始碼的一般用途政策引擎，可統一整個堆疊的政策執行作業。OPA 會使用類似 JSON 的語法設定文件，做為政策驗證的基準值。如需政策檢查的值範例，請參閱 [OPA 基準值](#)。
4. 隨機數與預期的 [nonce](#) (TLS [Exported Keying Material](#)) 相符。這項資訊已在上述 OPA 政策中驗證。

所有檢查完成並通過後，UWear 就能確認資料會安全地傳送及處理。接著，UWear 會透過相同的 TLS 工作階段回覆敏感的 PHI，USleep 就能使用這些資料計算顧客的睡眠品質。

2. 設定雲端資源

事前準備

1. 設定兩個 Google Cloud 專案，分別用於 USleep 和 UWear。如要进一步瞭解如何建立 Google Cloud 雲端專案，請參閱「[設定及瀏覽第一個 Google 專案](#)」程式碼研究室。如要瞭解如何擷取專案 ID，以及專案 ID 與專案名稱和專案編號有何不同，請參閱「[建立及管理專案](#)」。
2. 為專案[啟用帳單](#)。
3. 在其中一個 [Google 專案的 Cloud Shell](#) 中，設定必要的專案環境變數，如下所示。

```
export UWEAR_PROJECT_ID=<Google Cloud project id of UWear>
export USLEEP_PROJECT_ID=<Google Cloud project id of USleep>
```

4. 為兩個專案啟用機密運算 API 和下列 API。

```
gcloud config set project $UWEAR_PROJECT_ID
gcloud services enable \
  cloudapis.googleapis.com \
  cloudshell.googleapis.com \
  container.googleapis.com \
  containerregistry.googleapis.com \
  confidentialcomputing.googleapis.com

gcloud config set project $USLEEP_PROJECT_ID
gcloud services enable \
  cloudapis.googleapis.com \
  cloudshell.googleapis.com \
  container.googleapis.com \
  containerregistry.googleapis.com \
  confidentialcomputing.googleapis.com
```

5. 使用下列方式擷取主體 ID：

```
gcloud auth list

# Output should contain
# ACCOUNT: <Principal Identifier>

# Set your member variable
export MEMBER='user:<Principal Identifier>'
```

6. 為這兩個專案新增權限。如要新增權限，請按照「[授予 IAM 角色](#)」網頁上的詳細說明操作。

- 如要使用 `$UWEAR_PROJECT_ID`，您需要 [Artifact Registry 管理員](#)和[服務帳戶管理員](#)。

```
gcloud config set project $UWEAR_PROJECT_ID

# Add Artifact Registry Administrator role
gcloud projects add-iam-policy-binding $UWEAR_PROJECT_ID --member=$MEMBER --
role='roles/iam.serviceAccountAdmin'

# Add Service Account Administrator role
gcloud projects add-iam-policy-binding $UWEAR_PROJECT_ID --member=$MEMBER --
role='roles/artifactregistry.admin'
```

- 如要使用 `$USLEEP_PROJECT_ID`，您需要 [Compute 管理員](#)、[Storage 管理員](#)、[Artifact Registry 管理員](#) 和 [服務帳戶管理員](#)。

```
gcloud config set project $USLEEP_PROJECT_ID

# Add Service Account Administrator role
gcloud projects add-iam-policy-binding $USLEEP_PROJECT_ID --member=$MEMBER --
role='roles/iam.serviceAccountAdmin'

# Add Artifact Registry Administrator role
gcloud projects add-iam-policy-binding $USLEEP_PROJECT_ID --member=$MEMBER --
role='roles/artifactregistry.admin'

# Add Compute Administrator role
gcloud projects add-iam-policy-binding $USLEEP_PROJECT_ID --member=$MEMBER --
role='roles/compute.admin'

# Add Storage Administrator role
gcloud projects add-iam-policy-binding $USLEEP_PROJECT_ID --member=$MEMBER --
role='roles/compute.storageAdmin'
```

7. 在其中一個 Google Cloud 專案的 Cloud Shell 中，使用下列指令複製 [Confidential Space Codelab Github 存放區](#)，取得本程式碼研究室所需的指令碼。

```
git clone https://github.com/GoogleCloudPlatform/confidential-space.git
```

8. 將目錄變更為健康資料程式碼研究室的指令碼目錄。

```
cd confidential-space/codelabs/health_data_analysis_codelab/scripts
```

9. 更新 [config_env.sh](#) 指令碼中的這兩行，該指令碼位於 `codelabs/health_data_analysis_codelab/scripts` 目錄。將專案 ID 更新為 USleep 和 UWear 的專案 ID。請務必移除行首的註解符號「#」。

```
# TODO: Populate UWear and USleep Project IDs
export UWEAR_PROJECT_ID=your-uwear-project-id
export USLEEP_PROJECT_ID=your-usleep-project-id
```

10. 選用：設定任何預先存在的變數。您可以使用這些變數覆寫資源名稱 (例如 `export`

```
UWEAR_ARTIFACT_REPOSITORY='my-artifact-repository' )
```

- 您可以使用現有的雲端資源名稱設定下列變數。如果已設定變數，系統會使用專案中對應的現有雲端資源。如未設定變數，系統會根據 `config_env.sh` 指令碼中的值產生雲端資源名稱。

11. 執行 `config_env.sh` 指令碼，根據專案 ID 將資源名稱的其餘變數名稱設為值。

```
# Navigate to the scripts folder
cd ~/confidential-space/codelabs/health_data_analysis_codelab/scripts

# Run the config_env script
source config_env.sh

# Verify the variables were set
# Expected output for default variable should be `workload-sa`
echo $USLEEP_WORKLOAD_SERVICE_ACCOUNT
```

3. 下載根憑證

1. 為驗證認證服務傳回的獨立權杖，UWear 必須根據 Confidential Space 根憑證驗證簽章。UWear 必須[下載根憑證](#)，並儲存在本機。在其中一個 Google Cloud 專案的控制台中，執行下列指令：

```
cd ~/confidential-space/codelabs/health_data_analysis_codelab/src/uwear

wget https://confidentialcomputing.googleapis.com/.well-known/confidential_space_root.crt -O
confidential_space_root.pem
```

2. 產生下載的根憑證指紋

```
openssl x509 -fingerprint -in confidential_space_root.pem -noout
```

3. 確認指紋與下列 SHA-1 摘要相符：

```
B9:51:20:74:2C:24:E3:AA:34:04:2E:1C:3B:A3:AA:D2:8B:21:23:21
```

OIDC 與 PKI 權杖類型：

如要搭配使用 Confidential Space 與 Google Cloud 以外的受保護資源，信賴方可以要求使用 OIDC 或 PKI 權杖類型，並提供對象和/或隨機值。

可離線驗證的獨立權杖。離線驗證可提供更高的隱私權，且不必每次都擷取公開金鑰來驗證權杖，因此更實用。您可以使用先前下載的長期根憑證驗證權杖，不必進行線上通訊，並將根憑證做為可靠來源。

OIDC 權杖類型適用於其他雲端供應商，例如需要 OIDC ([OpenID Connect](#)) 權杖格式的 AWS 和 Azure。權杖會根據定期輪替的[公開金鑰](#)進行驗證，且效期較短，因此必須定期擷取。

步驟 4 · 約 0 分鐘

4. 建立工作負載服務帳戶

現在，您要建立兩個服務帳戶，分別用於 USleep 和 UWear 工作負載。執行 `create_service_accounts.sh` 指令碼，在 USleep 和 UWear 專案中建立工作負載服務帳戶。執行工作負載的 VM 會使用這些服務帳戶。

```
# Navigate to the scripts folder
cd ~/confidential-space/codelabs/health_data_analysis_codelab/scripts

# Run the create_service_accounts script
./create_service_accounts.sh
```

指令碼：

- 授予 `iam.serviceAccountUser` 角色，將服務帳戶附加至工作負載。
 - 將 `confidentialcomputing.workloadUser` 角色授予工作負載服務帳戶。這樣使用者帳戶就能產生認證權杖。
 - 將 `logging.logWriter` 角色授予工作負載服務帳戶權限。這樣一來，除了序列控制台，Confidential Space 環境也能將記錄寫入 Cloud Logging，因此 VM 終止後仍可存取記錄。建立工作負載
-

5. 建立 USleep 工作負載

在這個步驟中，您將為本程式碼研究室中使用的工作負載建立 Docker 映像檔。USleep 工作負載是簡單的 Golang 應用程式，可根據穿戴式裝置上的個人健康資訊判斷顧客的睡眠品質。

關於 USleep 工作負載

[USleep 工作負載](#)是簡單的 Golang 應用程式，可使用穿戴式裝置上的個人健康資訊判斷顧客的睡眠品質。USleep 工作負載主要有三個部分：

1. 設定 TLS 工作階段並擷取「匯出的金鑰材料」

```
func handleConnectionRequest(w http.ResponseWriter, r *http.Request) {
    // Upgrade HTTP Connection to a websocket.
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        fmt.Printf("failed to upgrade connection to a websocket with err: %v\n", err)
        return
    }
    defer conn.Close()

    // Get EKM
    hash, err := getEKMHashFromRequest(r)
    if err != nil {
        fmt.Printf("Failed to get EKM: %v", err)
    }
    ...
}

func getEKMHashFromRequest(r *http.Request) (string, error) {
    ekm, err := r.TLS.ExportKeyingMaterial("testing_nonce", nil, 32)
    if err != nil {
        err := fmt.Errorf("failed to get EKM from inbound http request: %w", err)
        return "", err
    }

    sha := sha256.New()
    sha.Write(ekm)
    hash := base64.StdEncoding.EncodeToString(sha.Sum(nil))

    fmt.Printf("EKM: %v\nSHA hash: %v", ekm, hash)
    return hash, nil
}
```

2. 向驗證服務要求權杖，並提供目標對象、Nonce 和 PKI 權杖類型。

```

func handleConnectionRequest(w http.ResponseWriter, r *http.Request) {
    ...

    // Request token with TLS Exported Keying Material (EKM) hashed.
    token, err := getCustomToken(hash)
    if err != nil {
        fmt.Printf("failed to get custom token from token endpoint: %v", err)
        return
    }

    // Respond to the client with the token.
    conn.WriteMessage(websocket.TextMessage, token)

    ...
}

var (
    socketPath      = "/run/container_launcher/teeserver.sock"
    tokenEndpoint    = "http://localhost/v1/token"
    contentType      = "application/json"
)

func getCustomToken(nonce string) ([]byte, error) {
    httpClient := http.Client{
        Transport: &http.Transport{
            // Set the DialContext field to a function that creates
            // a new network connection to a Unix domain socket
            DialContext: func(_ context.Context, _, _ string) (net.Conn, error) {
                return net.Dial("unix", socketPath)
            },
        },
    }

    body := fmt.Sprintf(`{
        "audience": "uwear",
        "nonces": ["%s"],
        "token_type": "PKI"
    }`, nonce)

    resp, err := httpClient.Post(tokenEndpoint, contentType, strings.NewReader(body))
    if err != nil {
        return nil, err
    }

    fmt.Printf("Response from launcher: %v\n", resp)
    text, err := io.ReadAll(resp.Body)
    if err != nil {
        return nil, fmt.Errorf("Failed to read resp.Body: %w", err)
    }

    fmt.Printf("Token from the attestation service: %s\n", text)
}

```

```
    return text, nil
}
```

3. 接收私密資料並計算使用者的睡眠品質

```
func handleConnectionRequest(w http.ResponseWriter, r *http.Request) {
    ...

    // Read the sensitive data
    _, content, err := conn.ReadMessage()
    if err != nil {
        fmt.Printf("failed to read message from the connection: %v\n", err)
    }
    fmt.Printf("Received content from other side, %v\n", string(content))

    // TODO: Handle sensitive data
    ...
}
```

建立 USleep 工作負載的步驟

1. 執行 `create_usleep_workload.sh` 指令碼，建立 USleep 工作負載。這個指令碼：
 - 建立 UWear 擁有的 Artifact Registry (`$USLEEP_ARTIFACT_REPOSITORY`)，工作負載會發布至該處。
 - 建構 `usleep/workload.go` 程式碼，並封裝至 Docker 映像檔。請參閱 USleep 的 [Dockerfile](#) 設定。
 - 將 Docker 映像檔發布至 UWear 擁有的 Artifact Registry (`$USLEEP_ARTIFACT_REPOSITORY`)。
 - 授予服務帳戶 Artifact Registry 的讀取權限 (`$USLEEP_WORKLOAD_SERVICE_ACCOUNT $USLEEP_ARTIFACT_REPOSITORY`)。

```
./create_usleep_workload.sh
```

2. 重要事項：在輸出記錄中，擷取 USleep 的映像檔摘要。

```
latest: digest: sha256:<USLEEP_IMAGE_DIGEST> size: 945
```

3. 前往 UWear 目錄

```
cd ~/confidential-space/codelabs/health_data_analysis_codelab/src/uwear
```

4. 將 `opa_validation_values.json` 中「`allowed_submods_container_image_digest`」下的值替換為 `USLEEP_IMAGE_DIGEST`。

```
# Replace the image digest
sed -i 's/sha256:bc4c32cb2ca046ba07dcd964b07a320b7d0ca88a5cf8e979da15cae68a2103ee/sha256:
<USLEEP_IMAGE_DIGEST>/' ~/confidential-
space/codelabs/health_data_analysis_codelab/src/uwear/opa_validation_values.json
```


步驟 6 · 約 0 分鐘

6. 建立 U Wear 工作負載

關於 U Wear 工作負載

U Wear 工作負載主要由 4 個部分組成：

1. 加入在 USleep 工作負載中建立的相同 TLS 工作階段，並透過安全的 TLS 工作階段從 USleep 擷取認證權杖。

```
func main() {
    fmt.Println("Initializing client...")

    tlsconfig := &tls.Config{
        // Skipping client verification of the server's certificate chain and host name since we
        are
        // doing custom verification using the attestation token.
        InsecureSkipVerify: true,
    }

    dialer := websocket.Dialer{
        TLSClientConfig:  tlsconfig,
        HandshakeTimeout: 5 * time.Second,
    }

    ipAddress := os.Getenv(ipAddrEnvVar)
    url := fmt.Sprintf("wss://%s:8081/connection", ipAddress)

    fmt.Printf("Attempting to dial to url %v...\n", url)
    conn, _, err := dialer.Dial(url, nil)
    if err != nil {
        fmt.Printf("Failed to dial to url %s, err %v\n", url, err)
        return
    }

    defer conn.Close()

    tokenString, ekm, err := retrieveTokenAndEKMFromConn(conn)
    if err != nil {
        fmt.Printf("Failed to retrieve token and EKM from connection: %v\n", err)
        return
    }

    fmt.Printf("token: %v\n", tokenString)

    ...
}
```

2. 透過下列方式驗證獨立權杖：

- 檢查 x5c 聲明是否包含憑證鏈結，且該鏈結是否從葉憑證正確鏈結至中繼憑證，最後再鏈結至根憑證。
- 檢查權杖是否由 x5c 聲明中包含的分葉憑證簽署。

- 檢查下載 / 儲存的根憑證是否與 x5c 聲明中的根憑證相同。


```

func main() {
    ...

    token, err := validatePKIToken(tokenString)
    if err != nil {
        fmt.Printf("Failed to validate PKI token, err: %v\n.", err)
        return
    }
    fmt.Println("PKI token validated successfully")

    ...
}

// validatePKIToken validates the PKI token returned from the attestation service.
// It verifies the token the certificate chain and that the token is signed by Google
// Returns a jwt.Token or returns an error if invalid.
func validatePKIToken(attestationToken string) (jwt.Token, error) {
    // IMPORTANT: The attestation token should be considered untrusted until the certificate
    chain and
    // the signature is verified.
    rawRootCertificate, err := readFile(rootCertificateFile)
    if err != nil {
        return jwt.Token{}, fmt.Errorf("readFile(%v) - failed to read root certificate: %w",
rootCertificateFile, err)
    }

    storedRootCert, err := decodeAndParsePEMCertificate(string(rawRootCertificate))
    if err != nil {
        return jwt.Token{}, fmt.Errorf("DecodeAndParsePEMCertificate(string) - failed to decode
and parse root certificate: %w", err)
    }

    jwtHeaders, err := extractJWTHeaders(attestationToken)
    if err != nil {
        return jwt.Token{}, fmt.Errorf("ExtractJWTHeaders(token) - failed to extract JWT headers:
%w", err)
    }

    if jwtHeaders["alg"] != "RS256" {
        return jwt.Token{}, fmt.Errorf("ValidatePKIToken(attestationToken, ekm) - got Alg: %v,
want: %v", jwtHeaders["alg"], "RS256")
    }

    // Additional Check: Validate the ALG in the header matches the certificate SPKI.
    // https://datatracker.ietf.org/doc/html/rfc5280#section-4.1.2.7
    // This is included in Golang's jwt.Parse function

    x5cHeaders := jwtHeaders["x5c"].([]any)
    certificates, err := extractCertificatesFromX5CHeader(x5cHeaders)
    if err != nil {
        return jwt.Token{}, fmt.Errorf("ExtractCertificatesFromX5CHeader(x5cHeaders) returned
error: %w", err)
    }

```

```

}

// Verify the leaf certificate signature algorithm is an RSA key
if certificates.LeafCert.SignatureAlgorithm != x509.SHA256WithRSA {
    return jwt.Token{}, fmt.Errorf("leaf certificate signature algorithm is not
SHA256WithRSA")
}

// Verify the leaf certificate public key algorithm is RSA
if certificates.LeafCert.PublicKeyAlgorithm != x509.RSA {
    return jwt.Token{}, fmt.Errorf("leaf certificate public key algorithm is not RSA")
}

// Verify the storedRootCertificate is the same as the root certificate returned in the
token
// storedRootCertificate is downloaded from the confidential computing well known endpoint
// https://confidentialcomputing.googleapis.com/.well-known/attestation-pki-root
err = compareCertificates(*storedRootCert, *certificates.RootCert)
if err != nil {
    return jwt.Token{}, fmt.Errorf("failed to verify certificate chain: %w", err)
}

err = verifyCertificateChain(certificates)
if err != nil {
    return jwt.Token{}, fmt.Errorf("VerifyCertificateChain(CertificateChain) - error
verifying x5c chain: %v", err)
}

keyFunc := func(token *jwt.Token) (any, error) {
    return certificates.LeafCert.PublicKey, nil
}

verifiedJWT, err := jwt.Parse(attestationToken, keyFunc)
return *verifiedJWT, err
}

// verifyCertificateChain verifies the certificate chain from leaf to root.
// It also checks that all certificate lifetimes are valid.
func verifyCertificateChain(certificates CertificateChain) error {
    // Additional check: Verify that all certificates in the cert chain are valid.
    // Note: The *x509.Certificate Verify method in Golang already validates this but for
other coding
    // languages it is important to make sure the certificate lifetimes are checked.
    if isCertificateLifetimeValid(certificates.LeafCert) {
        return fmt.Errorf("leaf certificate is not valid")
    }

    if isCertificateLifetimeValid(certificates.IntermediateCert) {
        return fmt.Errorf("intermediate certificate is not valid")
    }
}

```

```

interPool := x509.NewCertPool()
interPool.AddCert(certificates.IntermediateCert)

if isCertificateLifetimeValid(certificates.RootCert) {
    return fmt.Errorf("root certificate is not valid")
}
rootPool := x509.NewCertPool()
rootPool.AddCert(certificates.RootCert)

_, err := certificates.LeafCert.Verify(x509.VerifyOptions{
    Intermediates: interPool,
    Roots:         rootPool,
    KeyUsages:     []x509.ExtKeyUsage{x509.ExtKeyUsageAny},
})

if err != nil {
    return fmt.Errorf("failed to verify certificate chain: %v", err)
}

return nil
}

```

3. U Wear 工作負載接著會檢查權杖中的工作負載測量聲明，是否符合 [OPA 政策](#) 中指定的屬性條件。OPA 是開放原始碼的一般用途政策引擎，可統一整個堆疊的政策執行作業。OPA 會使用類似 JSON 的語法設定文件，做為政策驗證的基準值。

```

func main() {
    ...

    err = validateClaimsAgainstOPAPolicy(token, ekm)
    if err != nil {
        fmt.Printf("Failed to validate claims against OPA policy: %v\n", err)
        return
    }

    fmt.Println("Validated token and claims. Sending sensitive data")

    ...
}

// validateClaimsAgainstOPAPolicy validates the claims in the JWT token against the OPA
policy.
func validateClaimsAgainstOPAPolicy(token jwt.Token, ekm string) error {
    data, err := os.ReadFile("opa_validation_values.json")
    authorized, err := evaluateOPAPolicy(context.Background(), token, ekm, string(data))
    if err != nil {
        fmt.Println("Error evaluating OPA policy:", err)
        return fmt.Errorf("failed to evaluate OPA policy: %w", err)
    }
    if !authorized {
        fmt.Println("Remote TEE's JWT failed policy check.")
        return fmt.Errorf("remote TEE's JWT failed policy check")
    }
    fmt.Println("JWT is authorized.")
    return nil
}

// evaluateOPAPolicy returns boolean indicating if OPA policy is satisfied or not, or error
if occurred
func evaluateOPAPolicy(ctx context.Context, token jwt.Token, ekm string, policyData string)
(bool, error) {
    var claims jwt.MapClaims
    var ok bool
    if claims, ok = token.Claims.(jwt.MapClaims); !ok {
        return false, fmt.Errorf("failed to get the claims from the JWT")
    }

    module := fmt.Sprintf(opaPolicy, ekm)

    var json map[string]any
    err := util.UnmarshalJSON([]byte(policyData), &json)
    store := inmem.NewFromObject(json)

    // Bind 'allow' to the value of the policy decision
    // Bind 'hw_verified', 'image_verified', 'audience_verified', 'nonce_verified' to
their respective policy evaluations
    query, err := rego.New(

```

```

        rego.Query(regoQuery), // Argument 1 (Query string)
        rego.Store(store),     // Argument 2 (Data store)
        rego.Module("confidential_space.rego", module), // Argument 3 (Policy module)
    ).PrepareForEval(ctx)

    if err != nil {
        fmt.Printf("Error creating query: %v\n", err)
        return false, err
    }

    fmt.Println("Performing OPA query evaluation...")
    results, err := query.Eval(ctx, rego.EvalInput(claims))

    if err != nil {
        fmt.Printf("Error evaluating OPA policy: %v\n", err)
        return false, err
    } else if len(results) == 0 {
        fmt.Println("Undefined result from evaluating OPA policy")
        return false, err
    } else if result, ok := results[0].Bindings["allow"].(bool); !ok {
        fmt.Printf("Unexpected result type: %v\n", ok)
        fmt.Printf("Result: %+v\n", result)
        return false, err
    }

    fmt.Println("OPA policy evaluation completed.")

    fmt.Println("OPA policy result values:")
    for key, value := range results[0].Bindings {
        fmt.Printf("[ %s ]: %v\n", key, value)
    }
    result := results[0].Bindings["allow"]
    if result == true {
        fmt.Println("Policy check PASSED")
        return true, nil
    }
    fmt.Println("Policy check FAILED")
    return false, nil
}

```

- 範例 [OPA 基準值](#)：

```
{
  "allowed_submods_container_image_digest": [
    "sha256:<USLEEP_IMAGE_DIGEST>"
  ],
  "allowed_hwmodel": [
    "GCP_INTEL_TDX",
    "GCP_SHIELDED_VM",
    "GCP_AMD_SEV_ES",
    "GCP_AMD_SEV"
  ],
  "allowed_aud": [
    "uwear"
  ],
  "allowed_issuer": [
    "https://confidentialcomputing.googleapis.com"
  ],
  "allowed_secboot": [
    true
  ],
  "allowed_sw_name": [
    "CONFIDENTIAL_SPACE"
  ]
}
```

- 以 Rego 編寫的 [OPA 政策範例](#)。

```

package confidential_space

import rego.v1

default allow := false
default hw_verified := false
default image_digest_verified := false
default audience_verified := false
default nonce_verified := false
default issuer_verified := false
default secboot_verified := false
default sw_name_verified := false

allow if {
    hw_verified
    image_digest_verified
    audience_verified
    nonce_verified
    issuer_verified
    secboot_verified
    sw_name_verified
}

hw_verified if input.hwmodel in data.allowed_hwmodel
image_digest_verified if input.submods.container.image_digest in
data.allowed_submods_container_image_digest
audience_verified if input.aud in data.allowed_aud
issuer_verified if input.iss in data.allowed_issuer
secboot_verified if input.secboot in data.allowed_secboot
sw_name_verified if input.swname in data.allowed_sw_name
nonce_verified if {
    input.eat_nonce == "%s"
}

```

- Rego 查詢範例。

```

regoQuery = "
    allow = data.confidential_space.allow;
    hw_verified = data.confidential_space.hw_verified;
    image_digest_verified = data.confidential_space.image_digest_verified;
    audience_verified = data.confidential_space.audience_verified;
    nonce_verified = data.confidential_space.nonce_verified;
    issuer_verified = data.confidential_space.issuer_verified;
    secboot_verified = data.confidential_space.secboot_verified;
    sw_name_verified = data.confidential_space.sw_name_verified
"

```

- 在 OPA 驗證期間，UWear 工作負載也會驗證隨機值是否與預期的「隨機值」(傳輸層安全標準：TLS「匯出的金鑰材料」- EKM) 相符。[OPA 政策會使用傳遞至政策評估工具的 EKM 驗證隨機碼](#)。

取得 EKM Hash 的範例程式碼：

```
func getEKMDHashFromConn(c *websocket.Conn) (string, error) {
    conn, ok := c.NetConn().(*tls.Conn)
    if !ok {
        return "", fmt.Errorf("failed to cast NetConn to *tls.Conn")
    }

    state := conn.ConnectionState()
    ekm, err := state.ExportKeyingMaterial("testing_nonce", nil, 32)
    if err != nil {
        return "", fmt.Errorf("failed to get EKM from TLS connection: %w", err)
    }

    sha := sha256.New()
    sha.Write(ekm)
    hash := base64.StdEncoding.EncodeToString(sha.Sum(nil))

    return hash, nil
}
```

- 所有檢查完成並通過後，UWear 就能確認資料會安全地傳送及處理。接著，UWear 會透過相同的 TLS 工作階段回覆敏感的 PHI，USleep 就能使用這些資料計算顧客的睡眠品質。

```
func main() {
    ...

    fmt.Println("Validated token and claims. Sending sensitive data")

    data, err := readFile(mySensitiveDataFile)
    if err != nil {
        fmt.Printf("Failed to read data from the file: %v\n", err)
    }

    conn.WriteMessage(websocket.BinaryMessage, data)
    fmt.Println("Sent payload. Closing the connection")
    conn.Close()

    ...
}
```

建立 USleep 工作負載的步驟

- 前往指令碼目錄

```
cd ~/confidential-space/codelabs/health_data_analysis_codelab/scripts
```

- 執行 `create_uwear_workload.sh` 指令碼，建立 UWear 工作負載：

- 建立 UWear 擁有的 Artifact Registry (`$UWEAR_ARTIFACT_REPOSITORY`)，工作負載會發布至該處。
- 建構 `uwear/workload.go` 程式碼，並封裝至 Docker 映像檔。請參閱 USleep 的 [Dockerfile](#) 設定。

- 將 Docker 映像檔發布至 UWear 擁有的 Artifact Registry (`$UWEAR_ARTIFACT_REPOSITORY`) 。
- 授予服務帳戶 Artifact Registry 的讀取權限
(`$UWEAR_WORKLOAD_SERVICE_ACCOUNT` `$UWEAR_ARTIFACT_REPOSITORY`) 。

```
./create_uwear_workload.sh
```

授權工作負載

UWear 會根據下列屬性，授權工作負載存取客戶資料：

- *內容*：已驗證的代碼
- *地點*：安全環境
- *對象*：值得信賴的運算子

UWear 會使用 [Open Policy Agent](#) (OPA) 根據這些規定強制執行存取政策。

- *內容*：最新 `$USLEEP_WORKLOAD_IMAGE_NAME` 已上傳至 `$USLEEP_ARTIFACT_REPOSITORY` 存放區。
 - *位置*：Confidential Space 受信任的執行環境會在完全支援的 Confidential Space VM 映像檔上執行。
 - *對象*：`$USLEEP_WORKLOAD_SERVICE_ACCOUNT` 服務帳戶。
-

7. 執行 USleep 和 UWear 工作負載

執行 USleep 工作負載

```
gcloud config set project $USLEEP_PROJECT_ID

gcloud compute instances create \
  --confidential-compute-type=SEV \
  --shielded-secure-boot \
  --maintenance-policy=MIGRATE \
  --scopes=cloud-platform --zone=${USLEEP_PROJECT_ZONE} \
  --image-project=confidential-space-images \
  --image-family=confidential-space \
  --service-
account=${USLEEP_WORKLOAD_SERVICE_ACCOUNT}@${USLEEP_PROJECT_ID}.iam.gserviceaccount.com \
  --metadata ^~^tee-image-reference=${USLEEP_PROJECT_REPOSITORY_REGION}-
docker.pkg.dev/${USLEEP_PROJECT_ID}/${USLEEP_ARTIFACT_REPOSITORY}/${USLEEP_WORKLOAD_IMAGE_NAME}:
${USLEEP_WORKLOAD_IMAGE_TAG}~tee-restart-policy=Never~tee-container-log-redirect=true
usleep
```

回應應會傳回 STATUS: RUNNING，且 EXTERNAL_IP 也應會傳回類似如下的內容：

```
NAME: usleep
ZONE: us-west1-b
MACHINE_TYPE: n2d-standard-2
PREEMPTIBLE:
INTERNAL_IP: 10.138.0.6
EXTERNAL_IP: 34.168.56.10
STATUS: RUNNING
```

將外部 IP 儲存在變數中

```
export USLEEP_EXTERNAL_IP=<add your external IP>
```

注意：執行個體可使用兩個映像檔系列：`confidential-space-debug` 和 `confidential-space`。您可以選擇其中一種，進一步瞭解 [Confidential Space 映像檔](#)。

確認 USleep 工作負載是否正常執行

如要確認 USleep 工作負載是否正常運作，請前往 USleep 專案的「VM Instances」(VM 執行個體) 頁面。按一下「usleep」執行個體，然後按下「記錄」專區下方的「序列埠 1(主控台)」。伺服器啟動並執行後，記錄檔底部應會顯示類似下列的內容。

```
2024/09/13 17:00:00 workload task started
#####----- Local IP Address is <YOUR-LOCAL-IP> -----#####
Starting Server..
```

執行 UWear 工作負載

```
gcloud config set project $UWEAR_PROJECT_ID

gcloud compute instances create \
  --confidential-compute-type=SEV \
  --shielded-secure-boot \
  --maintenance-policy=MIGRATE \
  --scopes=cloud-platform --zone=${UWEAR_PROJECT_ZONE} \
  --image-project=confidential-space-images \
  --image-family=confidential-space \
  --service-
account=${UWEAR_WORKLOAD_SERVICE_ACCOUNT}@${UWEAR_PROJECT_ID}.iam.gserviceaccount.com \
  --metadata ^~^tee-image-reference=${UWEAR_PROJECT_REPOSITORY_REGION}-
docker.pkg.dev/${UWEAR_PROJECT_ID}/${UWEAR_ARTIFACT_REPOSITORY}/${UWEAR_WORKLOAD_IMAGE_NAME}:
${UWEAR_WORKLOAD_IMAGE_TAG}~tee-restart-policy=Never~tee-container-log-redirect=true~tee-env-
remote_ip_addr=$USLEEP_EXTERNAL_IP uwear
```

確認 UWear 工作負載是否正常執行

如要查看 UWear 工作負載的記錄，請前往 UWear 專案中的「VM Instances」(VM 執行個體) 頁面。按一下「uwear」執行個體，然後按下「記錄」專區下方的「序列埠 1(主控台)」。

執行個體完全啟動後，記錄輸出內容應如下所示

在 UWear 專案中，序號記錄應會顯示類似下列內容：

```
token: eyJ[... ]MrXUg
PKI token validated successfully
Performing OPA query evaluation...
OPA policy evaluation completed.
OPA policy result values:
[ hw_verified ]: true
[ image__digest_verified ]: true
[ audience_verified ]: true
[ nonce_verified ]: true
[ issuer_verified ]: true
[ secboot_verified ]: true
[ sw_name_verified ]: true
[ allow ]: true
Policy check PASSED
JWT is authorized.
Validated token and claims. Sending sensitive data
Sent payload. Closing the connection
```

如果 UWear 工作負載不是這樣，請參閱下方的附註瞭解操作說明。

注意：如果 UWear 工作負載遇到「Failed to dial to url [...] i/o timeout」(無法撥號至網址 [...] i/o 逾時) 錯誤，請嘗試更新[防火牆設定](#)，允許 USleep 專案的輸入流量。暫時允許來自來源 IP 0.0.0.0/0 的所有連入流量，將優先順序設為 1000，並允許所有通訊埠，方法是執行下列指令：

```
gcloud compute firewall-rules create <MY-RULE-NAME> --allow tcp
```

您可以暫時允許所有輸出流量，對 UWear 專案使用相同程序。

注意：如果工作負載遇到「Invalid value for field "resource.networkInterfaces[0].network". 請嘗試執行這項指令來建立 VPC。

```
gcloud compute networks create default \
```

```
--subnet-mode=auto \
```

```
--bgp-routing-mode=regional \
```

```
--mtu=1460
```

```
gcloud compute --project=$USLEEP_PROJECT_ID firewall-rules create allow-all-traffic --  
direction=INGRESS --priority=1000 --network=default --action=ALLOW --rules=tcp:8081 --  
source-ranges=0.0.0.0/0
```

查看 USleep 結果

如要查看結果，請返回 USleep 專案的「VM Instances」(VM 執行個體) 頁面。按一下「usleep」執行個體，然後按下「記錄」專區下方的「序列埠 1(主控台)」。在記錄底部查看工作負載的結果。應與下方範例類似。

```
Token from the attestation service: eyJhbGci...Ii5A3CJBuDM2o5Q
```

```
Received content from other side, {
```

```
  "name": "Amy",
```

```
  "age": 29,
```

```
  "sleep": {
```

```
    "light": {
```

```
      "minutes": 270
```

```
    },
```

```
    "deep": {
```

```
      "minutes": 135
```

```
    },
```

```
    "rem": {
```

```
      "minutes": 105
```

```
    }
```

```
  }
```

```
}
```

```
Sleep quality result: total sleep time is less than 8 hours
```

結果應為 "total sleep time is less than 8 hours".

恭喜！您已成功在 UWear 和 USleep 之間建立 Confidential Space，可分享機密資訊！

8. (選用) 執行未經授權的工作負載

在下一個情境中，USleep 會更新程式碼，並在 UWear 提供的睡眠資料上執行不同的工作負載。UWear 尚未同意這項新工作負載，且尚未更新 OPA 政策，因此無法使用新的圖片摘要。我們會驗證 UWear 不會將機密資料傳送至未經授權的工作負載。

USleep 修改工作負載

1. 將專案設為 \$USLEEP_PROJECT_ID。

```
gcloud config set project $USLEEP_PROJECT_ID
```

2. 刪除 USleep VM 執行個體。

```
gcloud compute instances delete usleep --zone $USLEEP_PROJECT_ZONE
```

3. 前往 usleep/workload.go 目錄。

```
cd ~/confidential-space/codelabs/health_data_analysis_codelab/src/usleep
```

4. 在 usleep/workload.go 檔案中，更新行 `"audience": "uwear"`。在這個範例中，如要變更圖片摘要，我們會將目標對象更新為 UWear 尚未核准的不同值。因此 UWear 應拒絕要求，原因有二：未經核准的圖片摘要和不正確的目標對象。

```
"audience": "anotherCompany.com",
```

5. 建立新的 USleep 工作負載

```
cd ~/confidential-space/codelabs/health_data_analysis_codelab/scripts  
  
./create_usleep_workload.sh
```

6. 建立新的 USleep VM 執行個體並執行工作負載

```
gcloud compute instances create \
  --confidential-compute-type=SEV \
  --shielded-secure-boot \
  --maintenance-policy=MIGRATE \
  --scopes=cloud-platform --zone=${USLEEP_PROJECT_ZONE} \
  --image-project=confidential-space-images \
  --image-family=confidential-space \
  --service-
account=${USLEEP_WORKLOAD_SERVICE_ACCOUNT}@${USLEEP_PROJECT_ID}.iam.gserviceaccount.com \
  --metadata ^~^tee-image-reference=${USLEEP_PROJECT_REPOSITORY_REGION}-
docker.pkg.dev/${USLEEP_PROJECT_ID}/${USLEEP_ARTIFACT_REPOSITORY}/${USLEEP_WORKLOAD_IMAGE_NAME}:
${USLEEP_WORKLOAD_IMAGE_TAG}~tee-restart-policy=Never~tee-container-log-redirect=true
usleep
```

7. 擷取新的 USleep 外部 IP，以供稍後使用

```
export USLEEP_EXTERNAL_IP=<add your external IP>
```

重新執行工作負載

8. 刪除 UWear VM 執行個體

```
gcloud config set project $UWEAR_PROJECT_ID

gcloud compute instances delete uwear --zone $UWEAR_PROJECT_ZONE
```

9. 使用新的外部 IP 重新建立 UWear VM 執行個體

```
gcloud compute instances create \
  --confidential-compute-type=SEV \
  --shielded-secure-boot \
  --maintenance-policy=MIGRATE \
  --scopes=cloud-platform --zone=${UWEAR_PROJECT_ZONE} \
  --image-project=confidential-space-images \
  --image-family=confidential-space \
  --service-
account=${UWEAR_WORKLOAD_SERVICE_ACCOUNT}@${UWEAR_PROJECT_ID}.iam.gserviceaccount.com \
  --metadata ^~^tee-image-reference=${UWEAR_PROJECT_REPOSITORY_REGION}-
docker.pkg.dev/${UWEAR_PROJECT_ID}/${UWEAR_ARTIFACT_REPOSITORY}/${UWEAR_WORKLOAD_IMAGE_NAME}:
${UWEAR_WORKLOAD_IMAGE_TAG}~tee-restart-policy=Never~tee-container-log-redirect=true~tee-env-
remote_ip_addr=${USLEEP_EXTERNAL_IP} uwear
```

10. 在 UWear 序列記錄中，應該會顯示下列訊息，且 USleep VM 不應收到任何私密資料

OPA policy result values:

```
[ nonce_verified ]: true  
[ issuer_verified ]: true  
[ secboot_verified ]: true  
[ sw_name_verified ]: true  
[ allow ]: false  
[ hw_verified ]: true  
[ image__digest_verified ]: false  
[ audience_verified ]: false
```

Policy check FAILED

Remote TEE's JWT failed policy check.

Failed to validate claims against OPA policy: remote TEE's JWT failed policy check

9. 清除

您可以使用[清理指令碼](#)，清除在本程式碼研究室中建立的資源。在這次清理作業中，系統會刪除下列資源：

- UWear 服務帳戶 (`$UWEAR_SERVICE_ACCOUNT`)。
- UWear 構件登錄檔 (`$UWEAR_ARTIFACT_REPOSITORY`)。
- UWear Compute 執行個體
- USleep 服務帳戶 (`$USLEEP_SERVICE_ACCOUNT`)。
- USleep 構件登錄檔 (`$USLEEP_ARTIFACT_REPOSITORY`)。
- USleep Compute 執行個體

```
./cleanup.sh
```

探索完畢後，請考慮按照[這些操作說明](#)刪除專案。

恭喜

恭喜，您已成功完成本程式碼研究室！

您瞭解如何使用 Confidential Space 安全地分享資料，同時確保資料機密性。

後續步驟

查看一些類似的程式碼研究室...

- [使用 Confidential Space 保護機器學習模型和智慧財產](#)
- [如何透過多方運算和機密空間進行數位資產交易](#)
- [使用 Confidential Space 分析機密資料](#)

其他資訊

- [感到孤單嗎？機密運算來救援](#)
 - [GCP 的機密運算](#)
 - [Confidential Space：保護隱私權的協作方式，引領未來趨勢](#)
 - [Google 和 Intel 如何提升機密運算的安全性](#)
 - [隱私權與進展：運用 Google Cloud 機密運算技術提升安全性](#)
-